Practical 12: MiniBank Collections Storage, Sorting & Concurrent Map

Problem Definition:

Upgrade MiniBank storage from a static array to the Collections Framework. Create AccountStore maintaining accounts in a HashMap keyed by account number, track transaction history in an ArrayList, provide sortedByBalance using a Comparator, implement Comparable on Account, and utilize ConcurrentHashMap in TransactionProcessor for thread-safe concurrent access.

Code:

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;

class Account implements Comparable<Account> {
    private final String accountNumber;
    private final String holderName;
    private double balance;

    public Account(String accountNumber, String holderName, double initialBalance) {
        this.accountNumber = accountNumber;
        this.holderName = holderName;
        this.balance = initialBalance;
    }

    public String getAccountNumber() {
        return accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }

    public double getBalance() {
        return balance;
    }

    public synchronized void deposit(double amount) {
        if (amount > 0) {
            this.balance += amount;
        }
    }

    @Override
    public int compareTo(Account other) {
        return this.accountNumber.compareTo(other.accountNumber);
    }

    @Override
    public String toString() {
        return String.format("Account[%s | %-12s | Balance: Rs. %.2f]", accountNumber,
holderName, balance);
```

```
    }
}

class AccountStore {
    private final Map<String, Account> accounts = new HashMap<>();
    private final List<String> transactionHistory = new ArrayList<>();

    public void add(Account account) {
        accounts.put(account.getAccountNumber(), account);
        transactionHistory.add("Registered account: " + account.getAccountNumber());
    }

    public Account get(String accountNumber) {
        return accounts.get(accountNumber);
    }

    public List<Account> getAll() {
        return new ArrayList<>(accounts.values());
    }

    public List<Account> sortedByBalance() {
        List<Account> sortedList = new ArrayList<>(accounts.values());
        sortedList.sort(Comparator.comparing(Account::getBalance, Comparator.reverseOrder()));
        return sortedList;
    }

    public List<String> getTransactionHistory() {
        return Collections.unmodifiableList(transactionHistory);
    }
}

class TransactionProcessor {
    private final ConcurrentHashMap<String, Account> sharedAccounts = new ConcurrentHashMap<>();

    public void registerAccount(Account acc) {
        sharedAccounts.put(acc.getAccountNumber(), acc);
    }

    public void executeConcurrentDeposit(String accNumber, double amount) {
        Account acc = sharedAccounts.get(accNumber);
        if (acc != null) {
            acc.deposit(amount);
        }
    }

    public Account getAccount(String accNumber) {
        return sharedAccounts.get(accNumber);
    }
}

public class PartB {
    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 12 - Part B: MiniBank Collections Storage, Sorting &
Concurrent Map");
        System.out.println();
```

```
        AccountStore store = new AccountStore();
        Account a1 = new Account("AC1001", "Rudra Modi", 5000.0);
        Account a2 = new Account("AC1002", "Aarav Patel", 2000.0);
        Account a3 = new Account("AC1003", "Diya Shah", 9000.0);
        Account a4 = new Account("AC1004", "Kabir Mehta", 15400.0);

        store.add(a1);
        store.add(a2);
        store.add(a3);
        store.add(a4);

        System.out.println("Direct Hash Lookup via get('AC1002'):");
        Account fetched = store.get("AC1002");
        System.out.println("  " + fetched);

        System.out.println();
        System.out.println("All Accounts (Natural Order by Account Number via Comparable):");
        List<Account> naturalOrderList = store.getAll();
        Collections.sort(naturalOrderList);
        for (Account a : naturalOrderList) {
            System.out.println("  " + a);
        }

        System.out.println();
        System.out.println("All Accounts (Sorted by Balance Descending via Comparator):");
        List<Account> sortedList = store.sortedByBalance();
        for (Account a : sortedList) {
            System.out.println("  " + a);
        }

        System.out.println();
        System.out.println("Initializing ConcurrentHashMap TransactionProcessor for Multi-
threading:");
        TransactionProcessor processor = new TransactionProcessor();
        processor.registerAccount(a1);
        processor.registerAccount(a3);
        processor.executeConcurrentDeposit("AC1001", 2500.0);
        System.out.println("  Updated AC1001 via Concurrent Processor: " +
processor.getAccount("AC1001"));
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-12\PartB.java
RUDRA MODI || 25DCS058

Practical 12 - Part B: MiniBank Collections Storage, Sorting & Concurrent Map

Direct Hash Lookup via get('AC1002'):
  Account[AC1002 | Aarav Patel  | Balance: Rs. 2000.00]

All Accounts (Natural Order by Account Number via Comparable):
  Account[AC1001 | Rudra Modi   | Balance: Rs. 5000.00]
  Account[AC1002 | Aarav Patel  | Balance: Rs. 2000.00]
  Account[AC1003 | Diya Shah    | Balance: Rs. 9000.00]
  Account[AC1004 | Kabir Mehta  | Balance: Rs. 15400.00]

All Accounts (Sorted by Balance Descending via Comparator):
  Account[AC1004 | Kabir Mehta  | Balance: Rs. 15400.00]
  Account[AC1003 | Diya Shah    | Balance: Rs. 9000.00]
  Account[AC1001 | Rudra Modi   | Balance: Rs. 5000.00]
  Account[AC1002 | Aarav Patel  | Balance: Rs. 2000.00]

Initializing ConcurrentHashMap TransactionProcessor for Multi-threading:
  Updated AC1001 via Concurrent Processor: Account[AC1001 | Rudra Modi   | Balance: Rs. 7500.00]
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: Why is AccountStore upgraded from a static array to HashMap<String, Account>?

Answer: A static array has fixed capacity, requires manual resizing, and incurs linear O(n) search time to look up an account by its account number. Upgrading to HashMap<String, Account> keyed by unique account number provides instant O(1) average lookup, insertion, and deletion times, dynamic capacity expansion, and seamless integration with Java Collections framework.

Q2: Why does TransactionProcessor require a ConcurrentHashMap for multi-threaded account access?

Answer: Standard HashMap is unsynchronized and not thread-safe; concurrent structural modifications by multiple worker threads can cause data races, silent update loss, or infinite loops. ConcurrentHashMap uses fine-grained lock striping and compare-and-swap (CAS) operations to allow safe, highly concurrent read and write operations across multiple threads without synchronizing the entire map, guaranteeing high transaction throughput and thread safety.